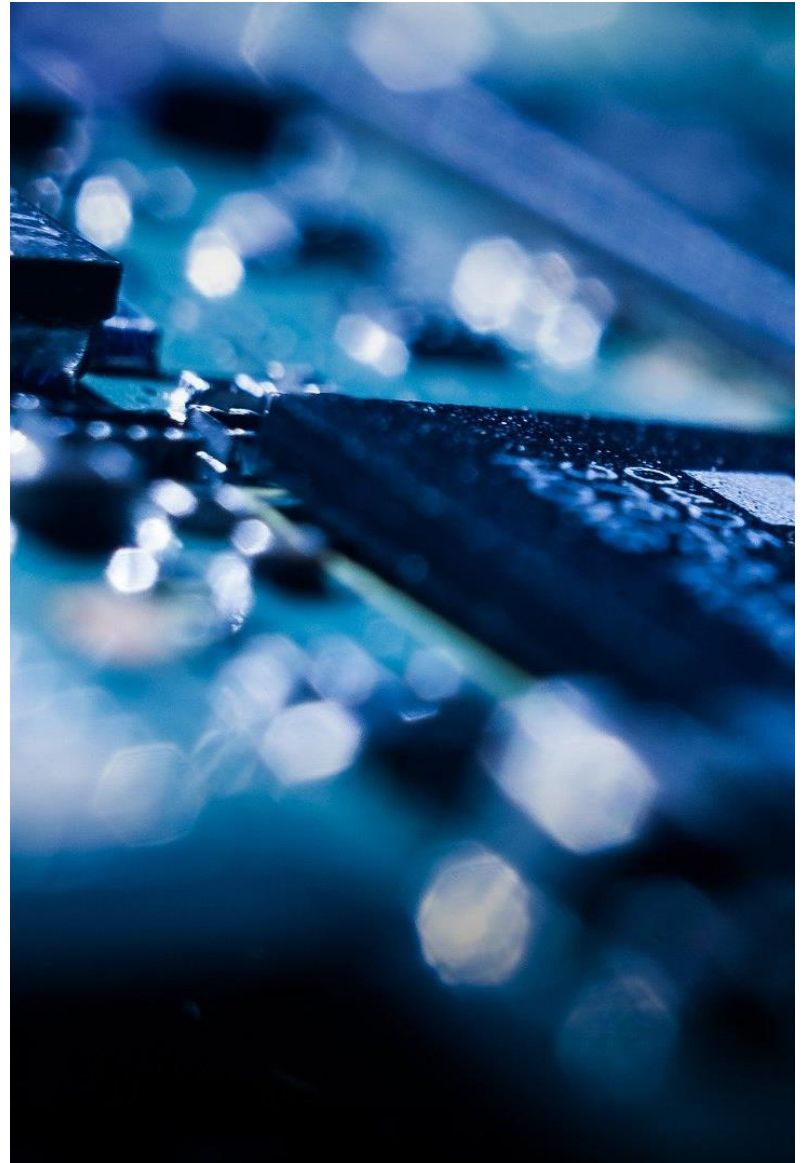


Programmierung eingebetteter Systeme

Peripherie



Prof. Dr. Henning Trsek

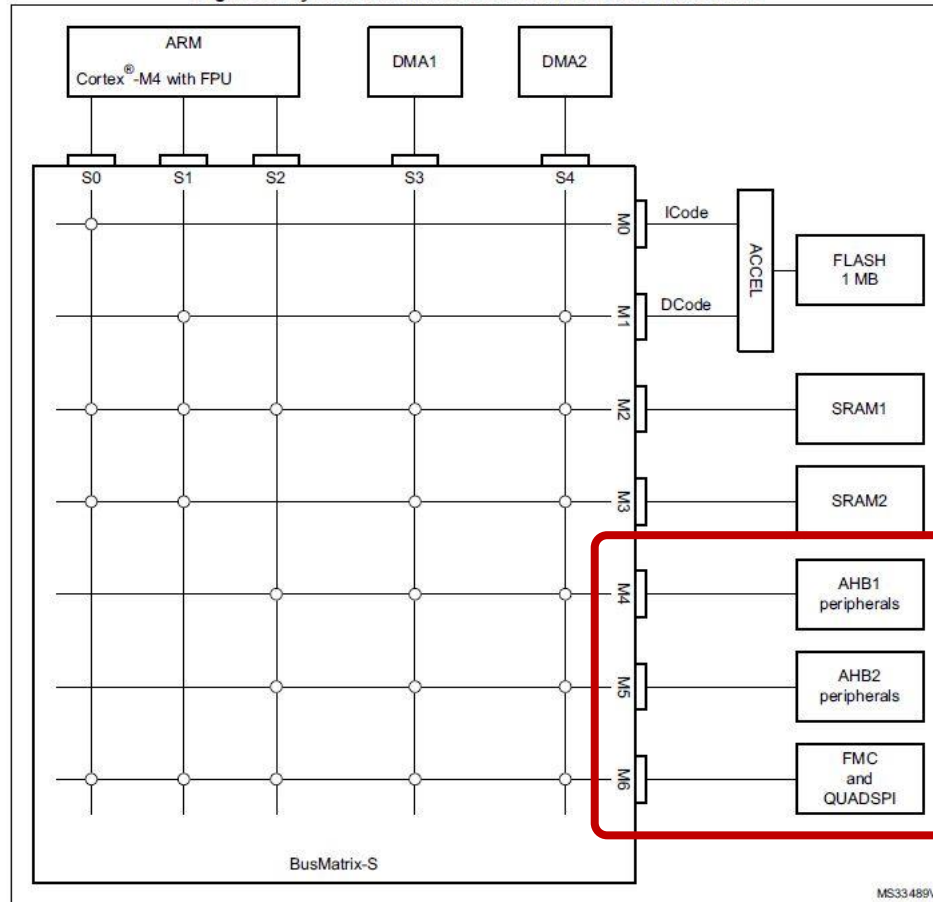
Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de

Überblick – Peripherie

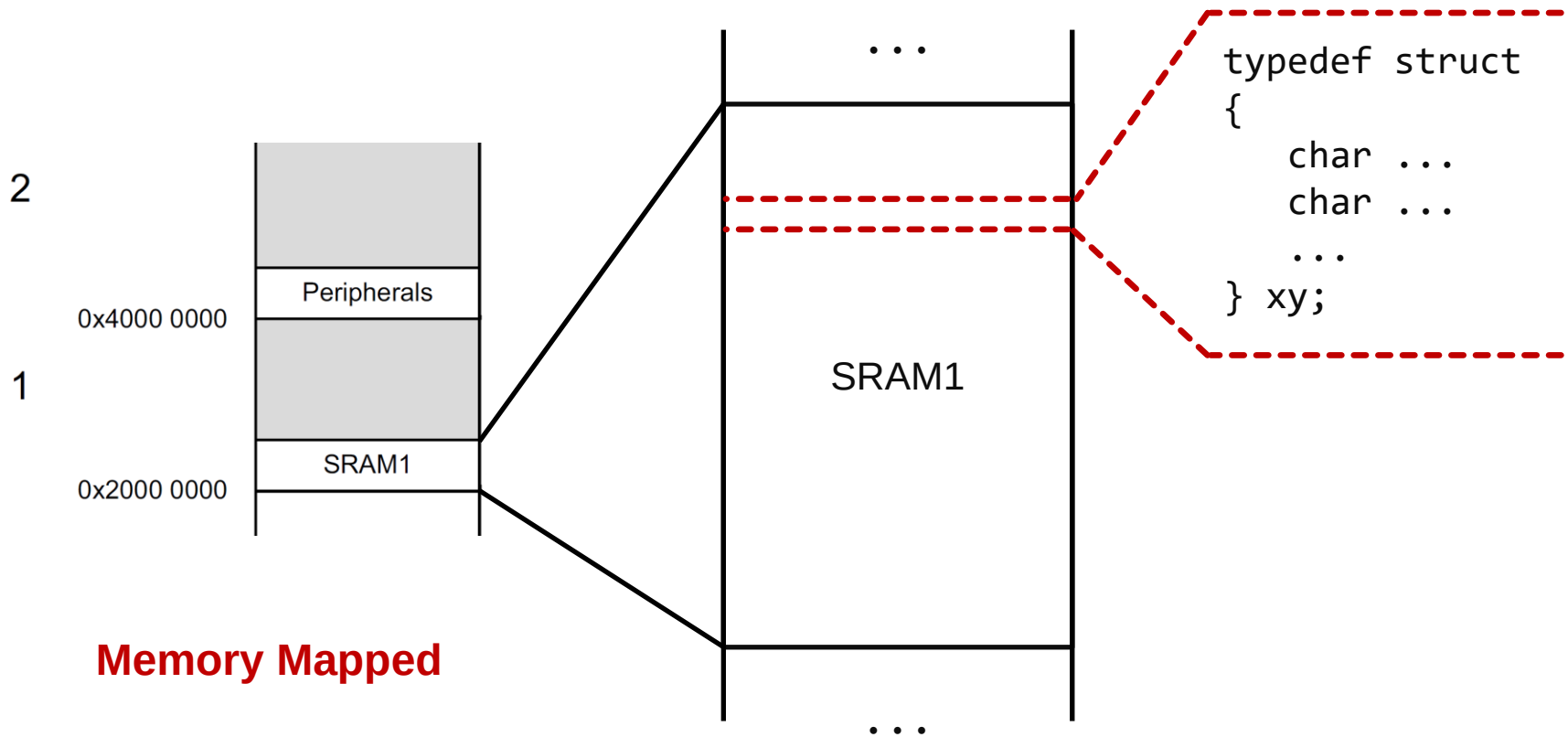
- ▶ Struktur-Prototypen und Peripheriebausteine
- ▶ Struktur-Prototypen für digitalen Input und Output
- ▶ Verkettete Listen als Anwendung von Struktur-Masken
- ▶ Weitere Verwendung von Zeigern

Anwendung Strukturprototypen: Peripherie

Figure 1. System architecture for STM32L47x/L48x devices



Anwendung Strukturprototypen: Peripherie



Anwendung Strukturprototypen: Peripherie

Bus	Boundary address	Size (bytes)	Peripheral
AHB2	0x5006 0800 - 0x5006 0BFF	1 KB	RNG
	0x5006 0400 - 0x5006 07FF	1 KB	Reserved
	0x5006 0000 - 0x5006 03FF	1 KB	AES ⁽²⁾
	0x5004 0400 - 0x5005 FFFF	127 KB	Reserved
	0x5004 0000 - 0x5004 03FF	1 KB	ADC
	0x5000 0000 - 0x5003 FFFF	256 KB	OTG_FS ⁽³⁾
	0x4800 2000 - 0x4FFF FFFF	~127 MB	Reserved
	0x4800 1C00 - 0x4800 1FFF	1 KB	GPIOH
	0x4800 1800 - 0x4800 1BFF	1 KB	GPIOG
	0x4800 1400 - 0x4800 17FF	1 KB	GPIOF
	0x4800 1000 - 0x4800 13FF	1 KB	GPIOE
	0x4800 0C00 - 0x4800 0FFF	1 KB	GPIOD
	0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC

Offset	Register name	31	30	29	28
0x14	GPIOx_ODR (where x = A..I)	Res.	Res.	Res.	Res.
	Reset value				
0x18	GPIOx_BSRR (where x = A..I)	BR15	BR14	BR13	BR12
	Reset value	0	0	0	0
0x1C	GPIOx_LCKR (where x = A..I)	Res.	Res.	Res.	Res.
	Reset value				
0x20	GPIOx_AFR1 (where x = A..I)	AFSEL7[3:0]			
	Reset value	0	0	0	0
0x24	GPIOx_AFRH (where x = A..I)	AFSEL15[3:0]			
	Reset value	0	0	0	0
0x28	GPIOx_BRR (where x = A..I)	Res.	Res.	Res.	Res.
	Reset value				
0x2C	GPIOx_ASCR (where x = A..H)	Res.	Res.	Res.	Res.
	Reset value				

Anwendung Strukturprototypen: Peripherie

```
typedef struct
{
    long gpiox_moder;    // 0x00
    long gpiox_otyper;   // 0x04
    long gpiox_ospeedr;  // 0x08
    long gpiox_pupdr;    // 0x0C
    long gpiox_idr;      // 0x10
    long gpiox_odr;      // 0x14
    long gpiox_bsrr;     // 0x18
    long gpiox_lckr;     // 0x1C
    long gpiox_afrl;     // 0x20
    long gpiox_afrh;     // 0x24
    long gpiox_brr;      // 0x28
    long gpiox_ascr;     // 0x2C
} GPIO_X;
```

Offset	Register name	31	30	29	28	27	26	25	24
--------	---------------	----	----	----	----	----	----	----	----

...

0x00	GPIOx_MODER (where x = C..I)	MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]	
	Reset value	1	1	1	1	1	1	1	1
0x04	GPIOx_OTYPER (where x = A..I)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value								
0x08	GPIOA_OSPEEDR	OSPEED15[1:0]		OSPEED14[1:0]		OSPEED13[1:0]		OSPEED12[1:0]	
	Reset value	0	0	0	0	1	1	0	0

...

0x14	GPIOx_ODR (where x = A..I)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value								
0x18	GPIOx_BSRR (where x = A..I)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8
	Reset value	0	0	0	0	0	0	0	0
0x1C	GPIOx_LCKR (where x = A..I)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value								
0x20	GPIOx_AFRL (where x = A..I)	AFSEL7[3:0]				AFSEL6[3:0]			
	Reset value	0	0	0	0	0	0	0	0

...

Struktur-Prototypen für digitalen Input und Output

Anwendung Strukturprototypen: Peripherie

```
#define ADDR_GPIOC 0x48000800
#define ADDR_GPIOB 0x48000400

typedef struct
{
    long gpiox_moder;    // 0x00
    long gpiox_otyper;   // 0x04
    long gpiox_ospeedr;  // 0x08
    long gpiox_pupdr;    // 0x0C
    long gpiox_idr;      // 0x10
    long gpiox_odr;      // 0x14
    long gpiox_bsrr;     // 0x18
    long gpiox_lckr;     // 0x1C
    long gpiox_afrl;     // 0x20
    long gpiox_afrh;     // 0x24
    long gpiox_brr;      // 0x28
    long gpiox_ascr;     // 0x2C
} GPIO_X;
```

```
GPIO_X* gpiob_ptr;
GPIO_X* gpioc_ptr;
long input;
long output;

gpiob_ptr = (GPIO_X*)ADDR_GPIOB;
gpioc_ptr = (GPIO_X*)ADDR_GPIOC;

input = gpiob_ptr->gpiox_idr;
output = (input >> 8);
gpioc_ptr->gpiox_odr = output;
```


Anwendung Strukturprototypen: Peripherie

```
#define ADDR_GPIOC 0x48000800  
#define ADDR_GPIOB 0x48000400
```

```
GPIO_X* gpiob_ptr;  
GPIO_X* gpioc_ptr;  
long input;  
long output;
```

```
typedef struct  
{  
    long gpiox_moder;  
    long gpiox_otyper;  
    long gpiox_ospeedr;  
    long gpiox_pupdr;  
    long gpiox_idr;  
    long gpiox_odr;  
    long gpiox_bsrr;  
    long gpiox_lckr;  
    long gpiox_afrrl;  
    long gpiox_afrh;  
    long gpiox_brr;  
    long gpiox_ascr;  
} GPIO_X;
```

0x4800 0C00 - 0x4800 0FFF	1 KB	GPIOD
0x4800 0800 - 0x4800 0BFF	1 KB	GPIOC
0x4800 0400 - 0x4800 07FF	1 KB	GPIOB
0x4800 0000 - 0x4800 03FF	1 KB	GPIOA

```
// 0x28  
// 0x2C
```

Anwendung Strukturprototypen: Peripherie

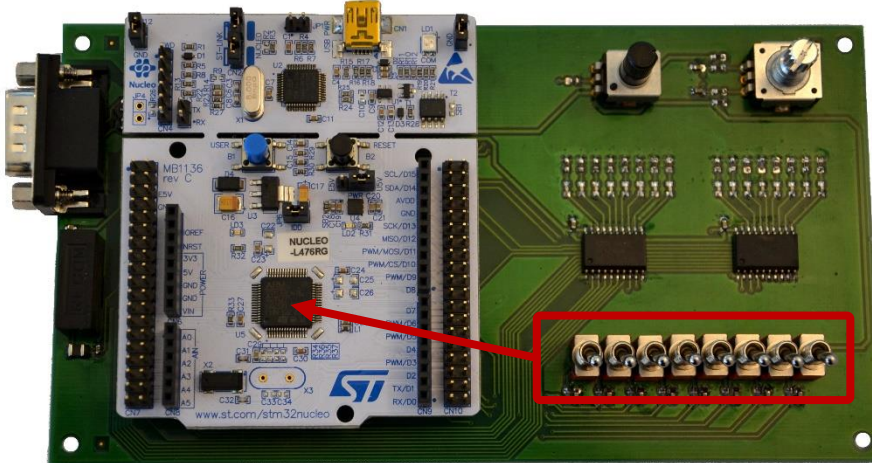
```
#define ADDR_GPIOC 0x48000800  
#define ADDR_GPIOB 0x48000400
```

```
typedef struct
```

```
GPIO_X* gpiob_ptr;  
GPIO_X* gpioc_ptr;  
long input;  
long output;
```

```
gpiob_ptr = (GPIO_X*)ADDR_GPIOB;  
gpioc_ptr = (GPIO_X*)ADDR_GPIOC;
```

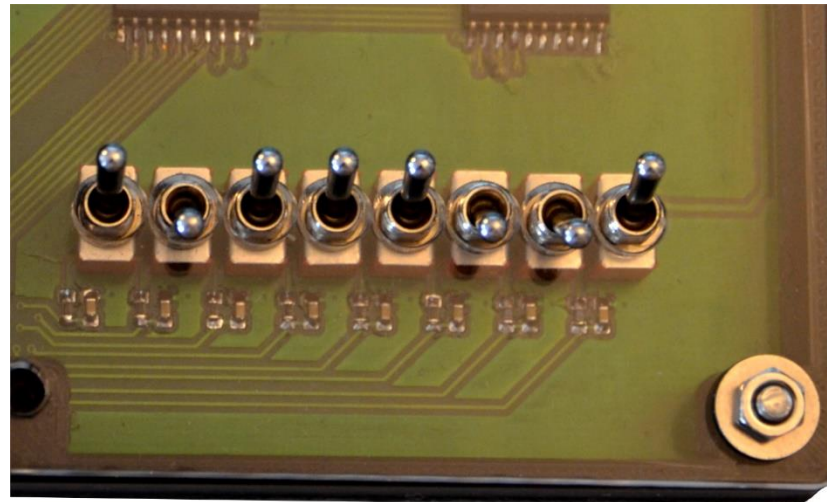
```
input = gpiob_ptr->gpiox_idr;  
output = (input >> 8);  
gpioc_ptr->gpiox_odr = output;
```



```
long gpiox_afrh;    // 0x24  
long gpiox_brr;     // 0x28  
long gpiox_ascr;    // 0x2C  
} GPIO_X;
```

Anwendung Strukturprototypen: Peripherie

8 Schalter:



1 0 1 1 1 0 0 1

Bit-Operatoren

&	bitweises UND	<code>var1 = var2 & var3;</code>
	bitweises ODER	<code>var1 = var2 var3;</code>
~	bitweises NICHT (Einernkomplement)	<code>var1 = ~var2;</code>
^	bitweises EXKLUSIV-ODER	<code>var1 = var1 ^ 0x01;</code>
>>	bitweises n-faches Rechtsschieben	<code>var1 = var2 >> n;</code>
<<	bitweises n-faches Linksschieben	<code>var1 = var2 << n;</code>

Bit-Operatoren

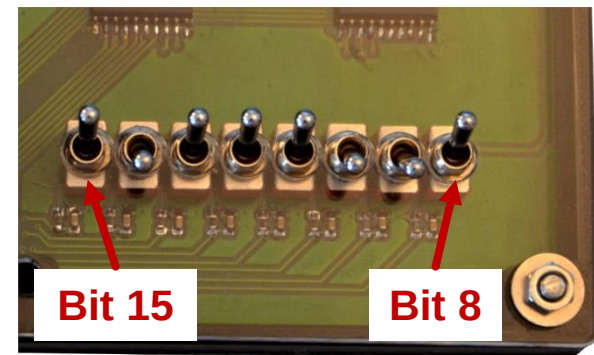
Beispiel 1: Bitgruppe 5, 6, 7 als Zahl weiterverarbeiten

```
GPIO_X* gpiob_ptr;  
GPIO_X* gpioc_ptr;  
long input;  
long output;  
  
gpiob_ptr = (GPIO_X*)ADDR_GPIOB;  
gpioc_ptr = (GPIO_X*)ADDR_GPIOC;  
  
input = gpiob_ptr->gpiox_idr;  
output = (input >> 8);  
gpioc_ptr->gpiox_odr = output;
```

Bit-Operatoren

Beispiel 1: Bitgruppe 5, 6, 7 als Zahl weiterverarbeiten

```
GPIO_X* gpiob_ptr;  
GPIO_X* gpioc_ptr;  
long input;  
long output;  
  
gpiob_ptr = (GPIO_X*)ADDR_GPIOB;  
gpioc_ptr = (GPIO_X*)ADDR_GPIOC;  
  
input = gpiob_ptr->gpiox_idr;  
output = (input >> 8);  
gpioc_ptr->gpiox_odr = output;
```



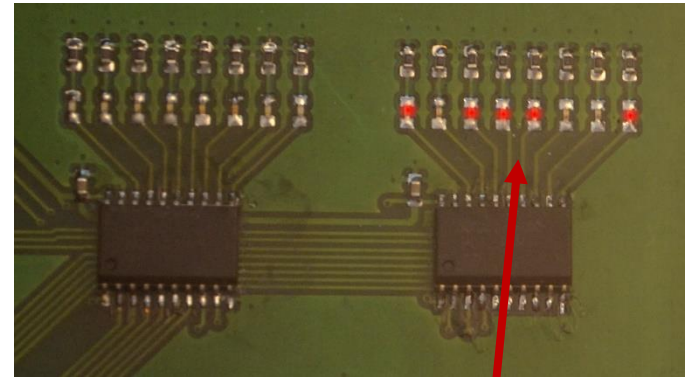
input:

1011100100000000

Bit-Operatoren

Beispiel 1: Bitgruppe 5, 6, 7 als Zahl weiter verarbeiten

```
GPIO_X* gpiob_ptr;  
GPIO_X* gpioc_ptr;  
long input;  
long output;  
  
gpiob_ptr = (GPIO_X*)ADDR_GPIOB;  
gpioc_ptr = (GPIO_X*)ADDR_GPIOC;  
  
input = gpiob_ptr->gpiox_idr;  
output = (input >> 8);  
gpioc_ptr->gpiox_odr = output;
```



output:

00000000**10111001**

Bit-Operatoren

Beispiel 2: Bitgruppe 8, 9, 10, 11 als Zahl weiter verarbeiten

```
#define BIT_MASK 0x0F00
```

```
GPIO_X* gpiob_ptr;
```

```
GPIO_X* gpioc_ptr;
```

```
long input;
```

```
long output;
```

```
gpiob_ptr = (GPIO_X*)ADDR_GPIOB;
```

```
gpioc_ptr = (GPIO_X*)ADDR_GPIOC;
```

```
input = gpiob_ptr->gpiox_idr;
```

```
output = (input & BIT_MASK);
```

```
gpioc_ptr->gpiox_odr = output;
```


Bit-Operatoren

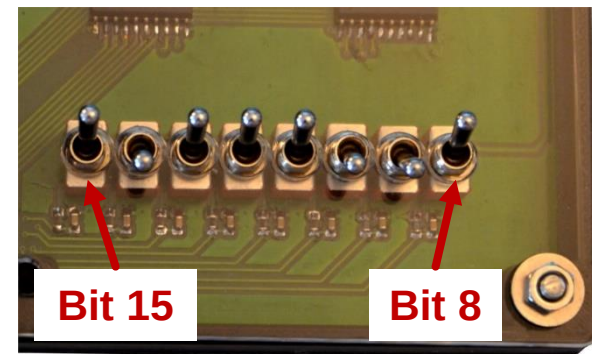
Beispiel 2: Bitgruppe 8, 9, 10, 11 als Zahl weiter verarbeiten

```
#define BIT_MASK 0x0F00
```

```
GPIO_X* gpiob_ptr;  
GPIO_X* gpior_ptr;  
long input;  
long output;
```

```
gpiob_ptr = (GPIO_X*)ADDR_GPIOB;  
gpior_ptr = (GPIO_X*)ADDR_GPIOR;
```

```
input = gpiob_ptr->gpiox_idr;  
output = (input & BIT_MASK);  
gpior_ptr->gpiox_odr = output;
```



input:

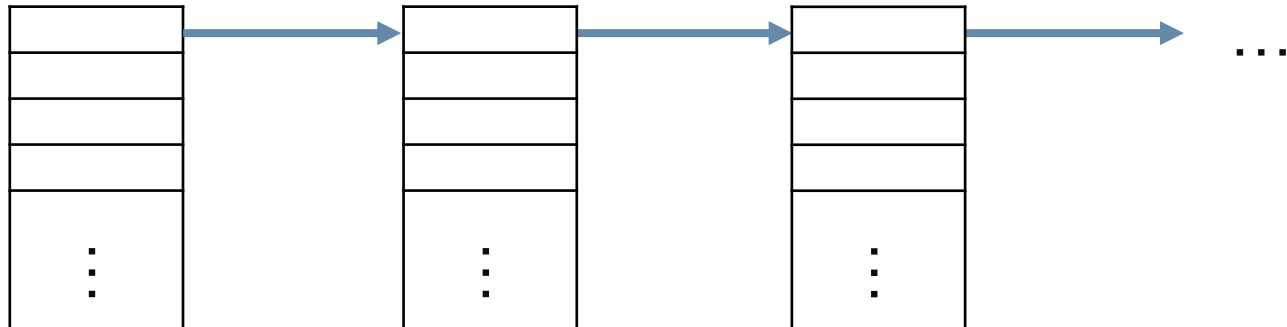
1011100100000000

Verkettete Listen als Anwendung von Struktur-Masken

Strukturen

Anwendung von Strukturmasken, verkettete Listen

Vorteile beim Umsortieren, Einketten neuer Elemente, Ausketten alter Elemente.
Daten (Inhalt der Strukturen) werden nicht kopiert.

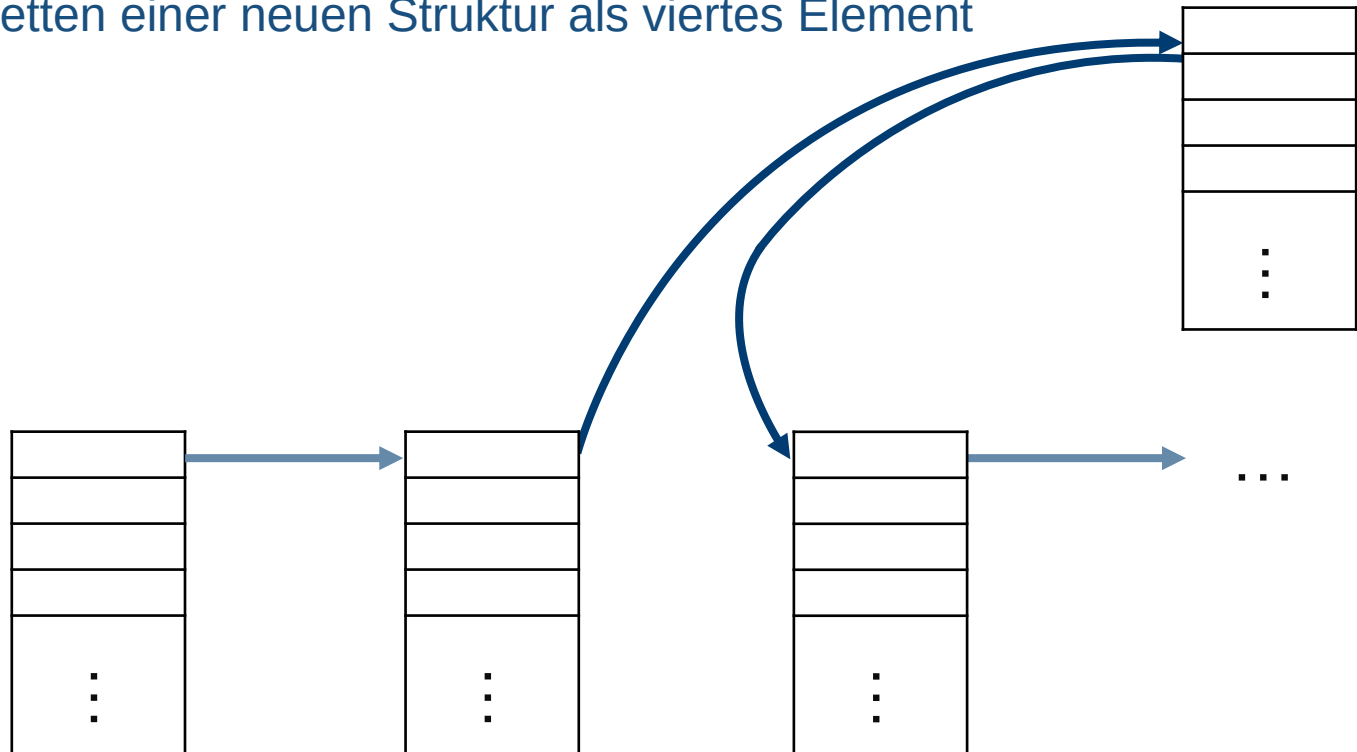


Strukturen

Anwendung von Strukturmasken, verkettete Listen

Vorteile beim Umsortieren, Einketten neuer Elemente, Ausketten alter Elemente.
Daten (Inhalt der Strukturen) werden nicht kopiert.

Beispiel: Einketten einer neuen Struktur als viertes Element



Strukturen

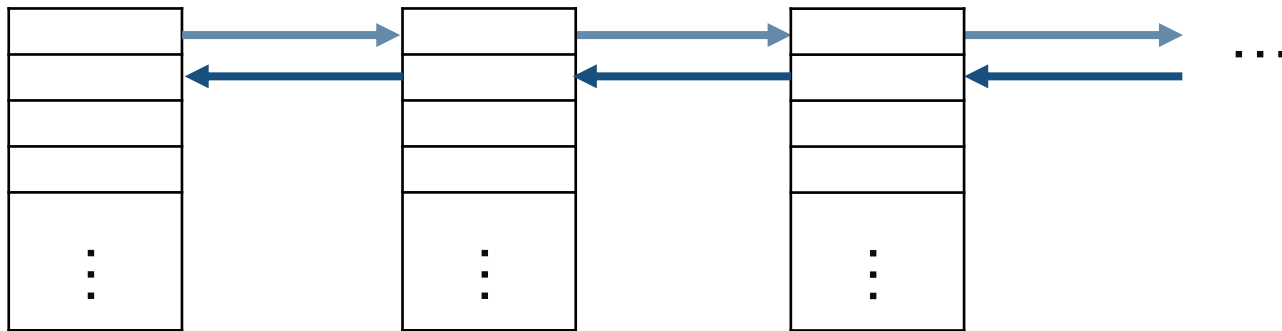
Anwendung von Strukturmasken, verkettete Listen

Felder aus Strukturen:

Logisch hintereinanderliegende Elemente (z.B. Index0 , Index1) liegen auch im Speicher hintereinander.

Verkettete Listen:

Logisch hintereinanderliegende Elemente können beliebig im Speicher verteilt sein. Verbindung durch Pointer.



Strukturen

Anwendung von Strukturmasken, verkettete Listen

```
// constant definitions
#define LIST_ORIGIN 0x20000000
#define MAX_LENGTH ...
...

// prototypes
typedef unsigned char Byte;
typedef struct liste
{
    struct liste *forward_ptr;
    int number;
    Byte value;
} Liste;
...

// modul-variables
Liste *list_ptr;
int counter;
...
```

```
void main(void)
{
    counter = 0;

    // install list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(loop = 1; loop < MAX_LENGTH; loop++)
    {
        list_ptr->forward_ptr = list_ptr + 1;
        list_ptr++;
    }
    list_ptr->forward_ptr = NULL;

    // use list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(...)
    {
        counter++;
        list_ptr->value = counter;
        list_ptr = list_ptr->forward_ptr;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen

Speicher

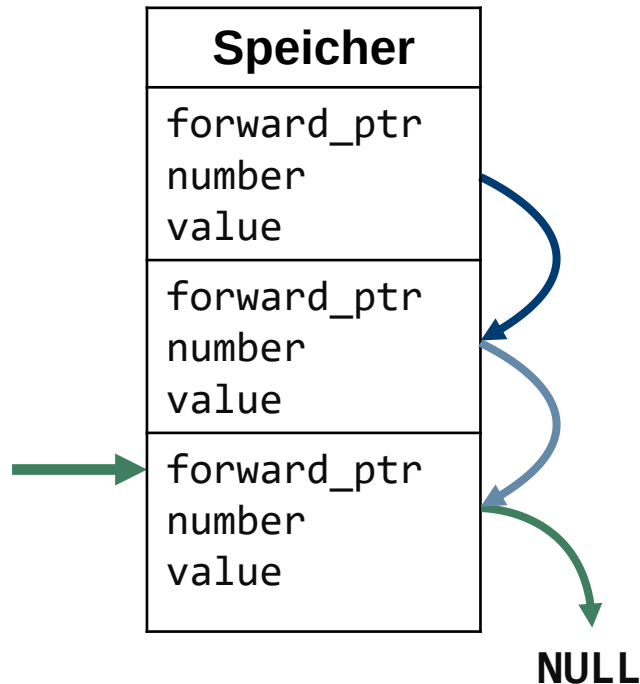
```
void main(void)
{
    counter = 0;

    // install list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(loop = 1; loop < MAX_LENGTH; loop++)
    {
        list_ptr->forward_ptr = list_ptr + 1;
        list_ptr++;
    }
    list_ptr->forward_ptr = NULL;

    // use list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(...)
    {
        counter++;
        list_ptr->value = counter;
        list_ptr = list_ptr->forward_ptr;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen



```
void main(void)
{
    counter = 0;

    // install list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(loop = 1; loop < MAX_LENGTH; loop++)
    {
        list_ptr->forward_ptr = list_ptr + 1;
        list_ptr++;
    }
    list_ptr->forward_ptr = NULL; Ende der Kette

    // use list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(...)
    {
        counter++;
        list_ptr->value = counter;
        list_ptr = list_ptr->forward_ptr;
    }
}
```


Strukturen

Anwendung von Strukturmasken, verkettete Listen

Speicher
forward_ptr number value
forward_ptr number value
forward_ptr number value

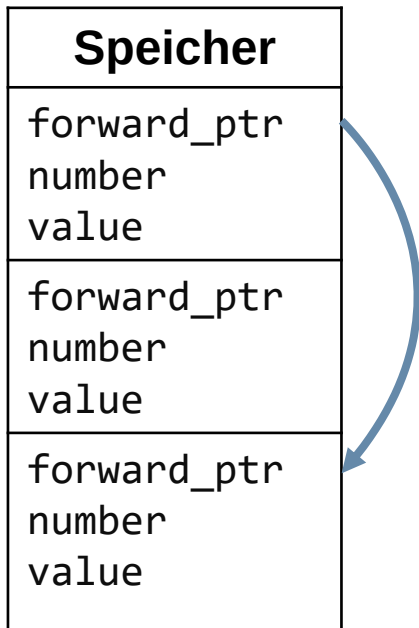
```
void main(void)
{
    counter = 0;

    // install list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(loop = 1; loop < MAX_LENGTH; loop++)
    {
        list_ptr->forward_ptr = list_ptr + 1;
        list_ptr++;
    }
    list_ptr->forward_ptr = NULL;

    // use list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(...)
    {
        counter++;
        list_ptr->value = counter;
        list_ptr = list_ptr->forward_ptr;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen



Die Listenelemente sind evtl. unsortiert. Nur die Pointer-Kette benutzen!

```
void main(void)
{
    counter = 0;

    // install list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(loop = 1; loop < MAX_LENGTH; loop++)
    {
        list_ptr->forward_ptr = list_ptr + 1;
        list_ptr++;
    }
    list_ptr->forward_ptr = NULL;

    // use list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(...)
    {
        counter++;
        list_ptr->value = counter;
        list_ptr = list_ptr->forward_ptr;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen

```
// constant definitions
#define LIST_ORIGIN 0x20000000
#define MAX_LENGTH ...
```

```
// prototypes
typedef unsigned char Byte;
typedef struct liste
{
    struct liste *forward_ptr;
    int number;
    Byte value;
} Liste;
```

```
// modul-variables
Liste *list_ptr;
int counter;
```

```
void main(void)
{
    counter = 0;

    // install list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(loop = 1; loop < MAX_LENGTH; loop++)
    {
        list_ptr->forward_ptr = list_ptr + 1;
        list_ptr++;
    }
    list_ptr->forward_ptr = NULL;

    // use list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(...)
    {
        counter++;
        list_ptr->value = counter;
        list_ptr = list_ptr->forward_ptr;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen

Ringpuffer

```
// constant definitions
#define LIST_ORIGIN 0x20000000
#define MAX_LENGTH ...
```

```
// prototypes
typedef unsigned char Byte;
typedef struct liste
{
    struct liste *forward_ptr;
    int number;
    Byte value;
} Liste;
```

```
// modul-variables
Liste *list_ptr;
int counter;
```

```
void main(void)
{
    counter = 0;
```

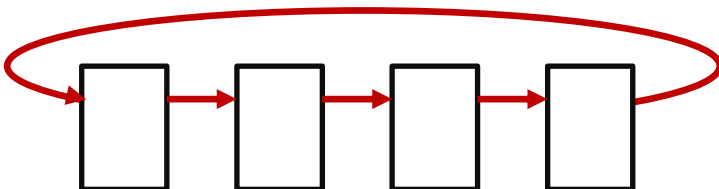
```
    // install list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(loop = 1; loop < MAX_LENGTH; loop++)
    {
        list_ptr->forward_ptr = list_ptr + 1;
        list_ptr++;
    }
```

```
    list_ptr->forward_ptr = NULL;
```

(Liste *)LIST_ORIGIN;

```
    // use list
    list_ptr = (Liste *)LIST_ORIGIN;
    for(...)
    {
        counter++;
        list_ptr->value = counter;
        list_ptr = list_ptr->forward_ptr;
    }
```

```
}
```



Strukturen

Anwendung von Strukturmasken, verkettete Listen

```
// prototypes
typedef unsigned char Byte;
typedef struct liste
{
    struct liste *forward_ptr;
    int number;
    Byte value;
} Liste;
```

```
// modul-variables
Liste *list_ptr_start;
Liste *list_ptr_end;
Liste *list_ptr_new;
```

```
void main(void)
{
    // install some list-members
    list_ptr_new = (Liste *)malloc(sizeof(Liste));
    list_ptr_start = list_ptr_new;
    list_ptr_end = list_ptr_new;
    ...

    list_ptr_new = (Liste*)malloc(sizeof(Liste));
    list_ptr_end->forward_ptr = list_ptr_new;
    list_ptr_end = list_ptr_new;

    if(...) // need new list-numbers
    {
        list_ptr_new = (Liste *)malloc(sizeof(Liste));
        list_ptr_end->forward_ptr = list_ptr_new;
        list_ptr_end = list_ptr_new;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen

```
// prototypes
typedef unsigned char Byte;
typedef struct liste
{
    struct liste *forward_ptr;
    int number;
    Byte value;
} Liste;
```

```
// modul-variables
Liste *list_ptr_start;
Liste *list_ptr_end;
Liste *list_ptr_new;
```

```
void main(void)
{
    // install some list-members
    list_ptr_new = (Liste *)malloc(sizeof(Liste));
    list_ptr_start = list_ptr_new;
    list_ptr_end = list_ptr_new;
    ...

    list_ptr_new = (Liste*)malloc(sizeof(Liste));
    list_ptr_end->forward_ptr = list_ptr_new;
    list_ptr_end = list_ptr_new;

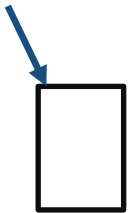
    if(...) // need new list-numbers
    {
        list_ptr_new = (Liste *)malloc(sizeof(Liste));
        list_ptr_end->forward_ptr = list_ptr_new;
        list_ptr_end = list_ptr_new;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen

```
// prototypes
typedef unsigned char Byte;
typedef struct liste
{
    struct liste *forward_ptr;
    int number;
    Byte value;
} Liste;

// modul-variables
Liste *list_ptr_start;
Liste *list_ptr_end;
Liste *list_ptr_new;
```



```
void main(void)
{
    // install some list-members
    → list_ptr_new = (Liste *)malloc(sizeof(Liste));
    list_ptr_start = list_ptr_new;
    list_ptr_end = list_ptr_new;
    ...

    list_ptr_new = (Liste*)malloc(sizeof(Liste));
    list_ptr_end->forward_ptr = list_ptr_new;
    list_ptr_end = list_ptr_new;

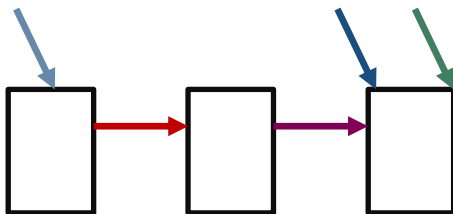
    if(...) // need new list-numbers
    {
        list_ptr_new = (Liste *)malloc(sizeof(Liste));
        list_ptr_end->forward_ptr = list_ptr_new;
        list_ptr_end = list_ptr_new;
    }
}
```

Strukturen

Anwendung von Strukturmasken, verkettete Listen

```
// prototypes
typedef unsigned char Byte;
typedef struct liste
{
    struct liste *forward_ptr;
    int number;
    Byte value;
} Liste;
```

```
// modul-variables
Liste *list_ptr_start;
Liste *list_ptr_end;
Liste *list_ptr_new;
```



```
void main(void)
{
    // install some list-members
    list_ptr_new = (Liste *)malloc(sizeof(Liste));
    → list_ptr_start = list_ptr_new;
    → list_ptr_end = list_ptr_new;
    ...

    → list_ptr_new = (Liste*)malloc(sizeof(Liste));
    → list_ptr_end->forward_ptr = list_ptr_new;
    → list_ptr_end = list_ptr_new;

    if(...) // need new list-numbers
    {
        → list_ptr_new = (Liste *)malloc(sizeof(Liste));
        → list_ptr_end->forward_ptr = list_ptr_new;
        → list_ptr_end = list_ptr_new;
    }
}
```

**Feld dynamisch
vergrößern**

Weitere Verwendung von Zeigern

Zeiger auf Zeiger

Initialisierung von GPIOs und Zeiger-Initialisierung im Unterprogramm

```
#define ADDR_GPIOC 0x48000800
#define ADDR_GPIOB 0x48000400

typedef struct
{
    long gpiox_moder;    // 0x00
    long gpiox_otyper;   // 0x04
    long gpiox_ospeedr;  // 0x08
    long gpiox_pupdr;    // 0x0C
    long gpiox_idr;      // 0x10
    long gpiox_odr;      // 0x14
    long gpiox_bsrr;     // 0x18
    long gpiox_lckr;     // 0x1C
    long gpiox_afrl;     // 0x20
    long gpiox_afrh;     // 0x24
    long gpiox_brr;      // 0x28
    long gpiox_ascr;     // 0x2C
} GPIO_X;
```

```
GPIO_X* gpiob_ptr;
GPIO_X* gpioc_ptr;
long input;
long output;
```

void main(void) **Parameterübergabe?**

```
{
    gpiob_ptr = (GPIO_X*)ADDR_GPIOB;
    gpioc_ptr = (GPIO_X*)ADDR_GPIOC;
    ...

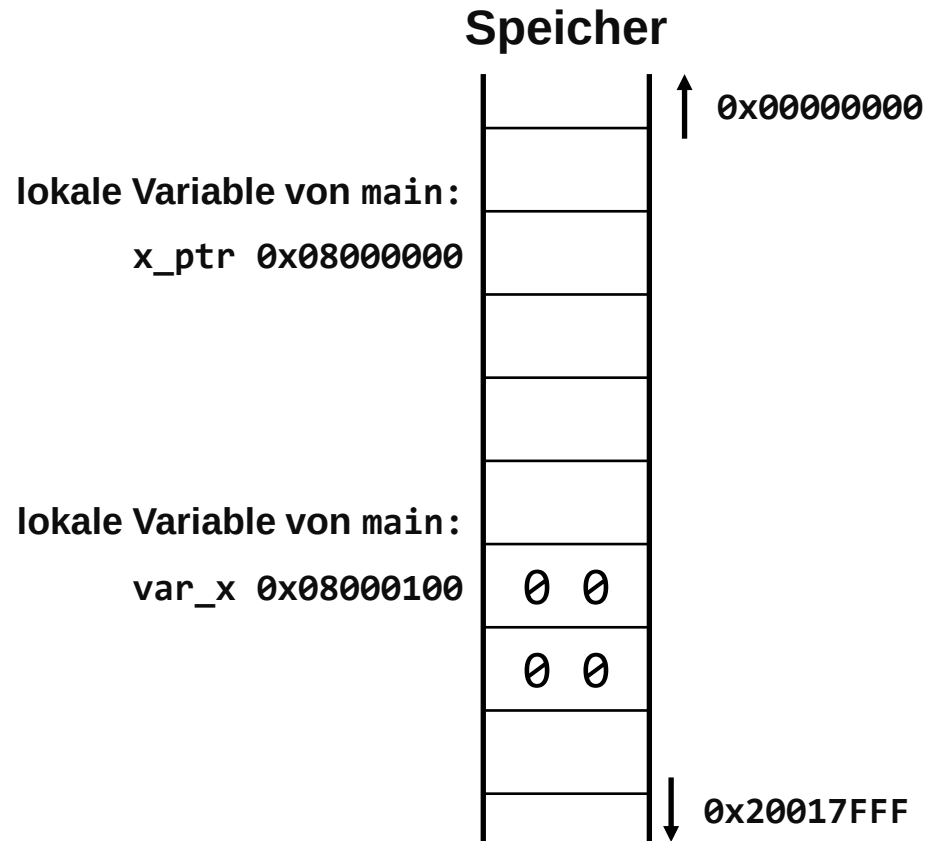
    while (1)
    {
        input = gpiob_ptr->gpiox_idr;
        input = (input & MASK_B);
        output = (input >> 8);
        gpioc_ptr->gpiox_odr = output;
        ...
    }
}
```

Zeiger auf Zeiger

Beispiel: Parameterübergabe per Reference (Übergabe eines Pointers)

```
// Hauptprogramm
void main(void)
{
    int error;
    long int var_x;
    long int *x_ptr;
    x_ptr = &var_x;
    ...
    error = func1(x_ptr);
    ...
}

// Funktion
int func1(long int *f_ptr)
{
    ...
    *f_ptr = 10;
    ...
}
```



Zeiger auf Zeiger

Beispiel: Parameterübergabe per Reference (Übergabe eines Pointers)

```
// Hauptprogramm
void main(void)
{
    int error;
    long int var_x;
    long int *x_ptr;
    x_ptr = &var_x;
    ...
    error = func1(x_ptr);
    ...
}
```

```
// Funktion
int func1(long int *f_ptr)
{
    ...
    *f_ptr = 10;
    ...
}
```

Der Inhalt von x_ptr ist aus der Funktion nicht veränderbar (nur die Zahl 0x20000500 ist bekannt).

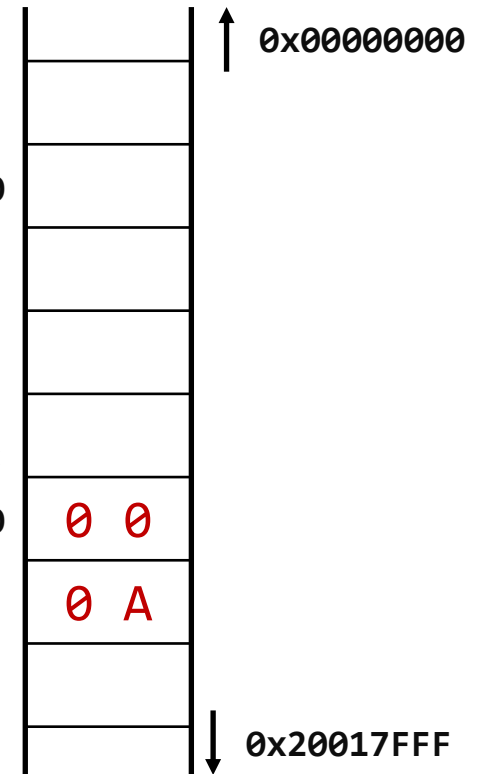
lokale Variable von main:

x_ptr 0x08000000

lokale Variable von main:

var_x 0x08000100

Speicher



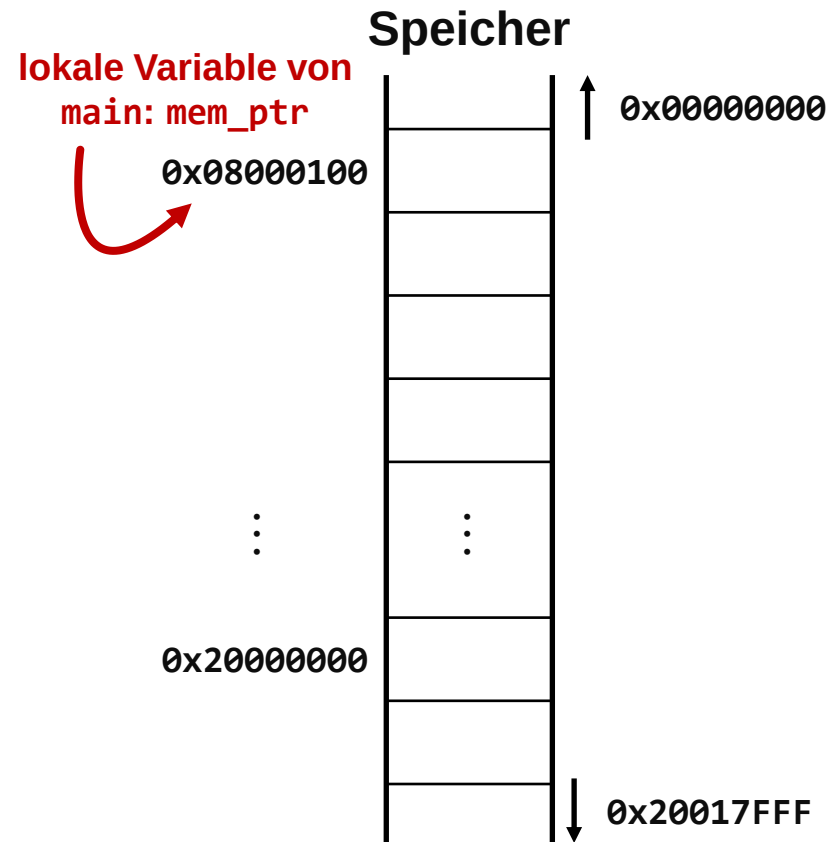
Zeiger auf Zeiger

Die Veränderung des Zeigers soll im aufrufenden Programm sichtbar sein:
Adresse des Zeigers übergeben.

```
#define MEM 0x20000000

// Hauptprogramm
void main(void)
{
    char* mem_ptr;
    func1(&mem_ptr);
    *mem_ptr = 0x03;
    ...
}

// Funktion
void func1(char** m_ptr)
{
    *m_ptr = (char*)MEM;
    ...
}
```



Zeiger auf Zeiger

Die Veränderung des Zeigers soll im aufrufenden Programm sichtbar sein:
Adresse des Zeigers übergeben.

```
#define MEM 0x20000000
```

```
// Hauptprogramm
```

```
void main(void)
```

```
{
```

```
    char* mem_ptr;
```

```
    func1(&mem_ptr);
```

```
    *mem_ptr = 0x03;
```

```
    ...
```

```
}
```

```
// Funktion
```

```
void func1(char** m_ptr)
```

```
{
```

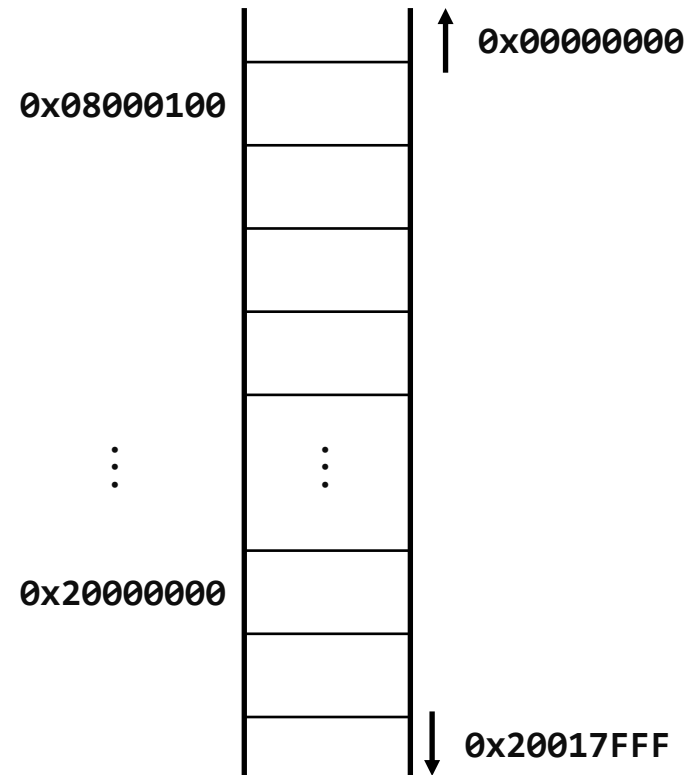
```
    *m_ptr = (char*)MEM;
```

```
    ...
```

```
}
```

Die Adresse des
Pointers (die Zahl
0x08000100) wird
an die Funktion
übergeben.

Speicher



Zeiger auf Zeiger

Die Veränderung des Zeigers soll im aufrufenden Programm sichtbar sein:
Adresse des Zeigers übergeben.

```
#define MEM 0x20000000
```

```
// Hauptprogramm
```

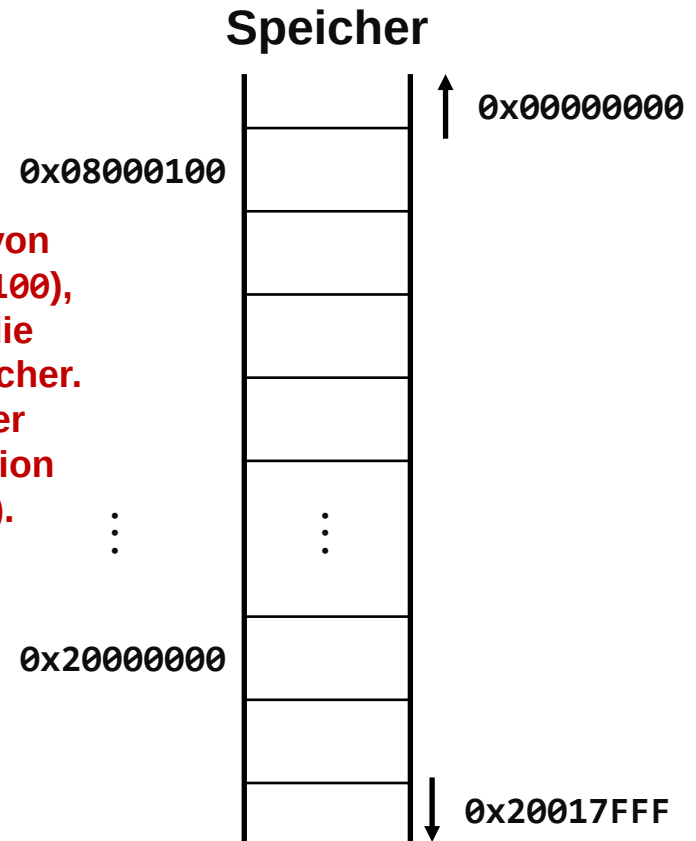
```
void main(void)
```

```
{  
    char* mem_ptr;  
    func1(&mem_ptr);  
    *mem_ptr = 0x03;  
    ...  
}
```

```
// Funktion
```

```
void func1(char** m_ptr)  
{  
    *m_ptr = (char*)MEM;  
    ...  
}
```

In `m_ptr` steht eine Kopie von
&`mem_ptr` (die Zahl `0x08000100`),
d.h. der Pointer zeigt auf die
Adresse `0x08000100` im Speicher.
`m_ptr` steht im Bereich der
lokalen Variablen der Funktion
(temporär auf dem Stack).



Zeiger auf Zeiger

Die Veränderung des Zeigers soll im aufrufenden Programm sichtbar sein:
Adresse des Zeigers übergeben.

```
#define MEM 0x20000000
```

```
// Hauptprogramm
```

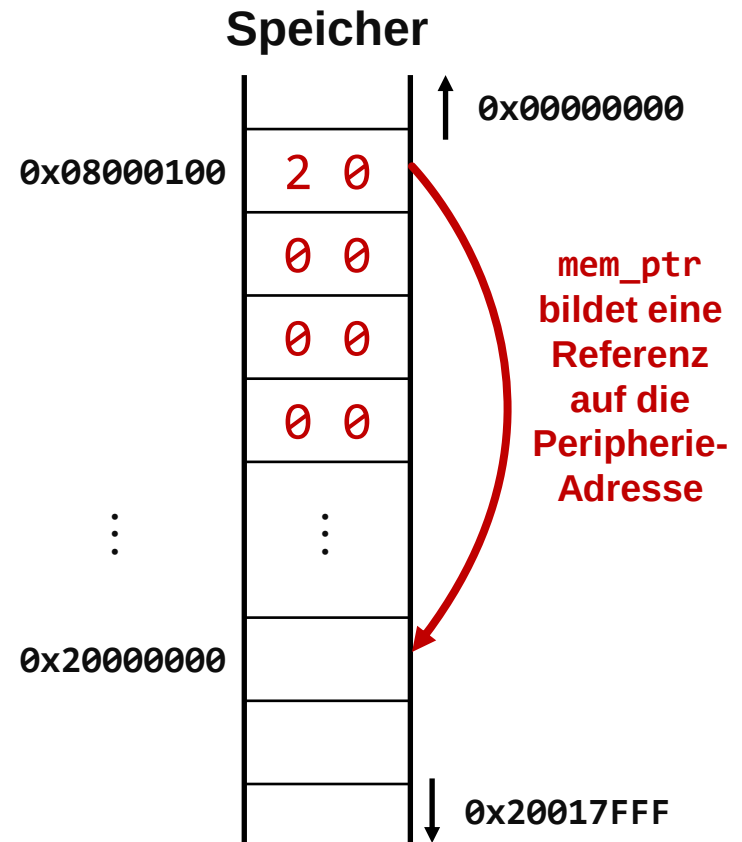
```
void main(void)
```

```
{  
    char* mem_ptr;  
    func1(&mem_ptr);  
    *mem_ptr = 0x03;  
    ...  
}
```

```
// Funktion
```

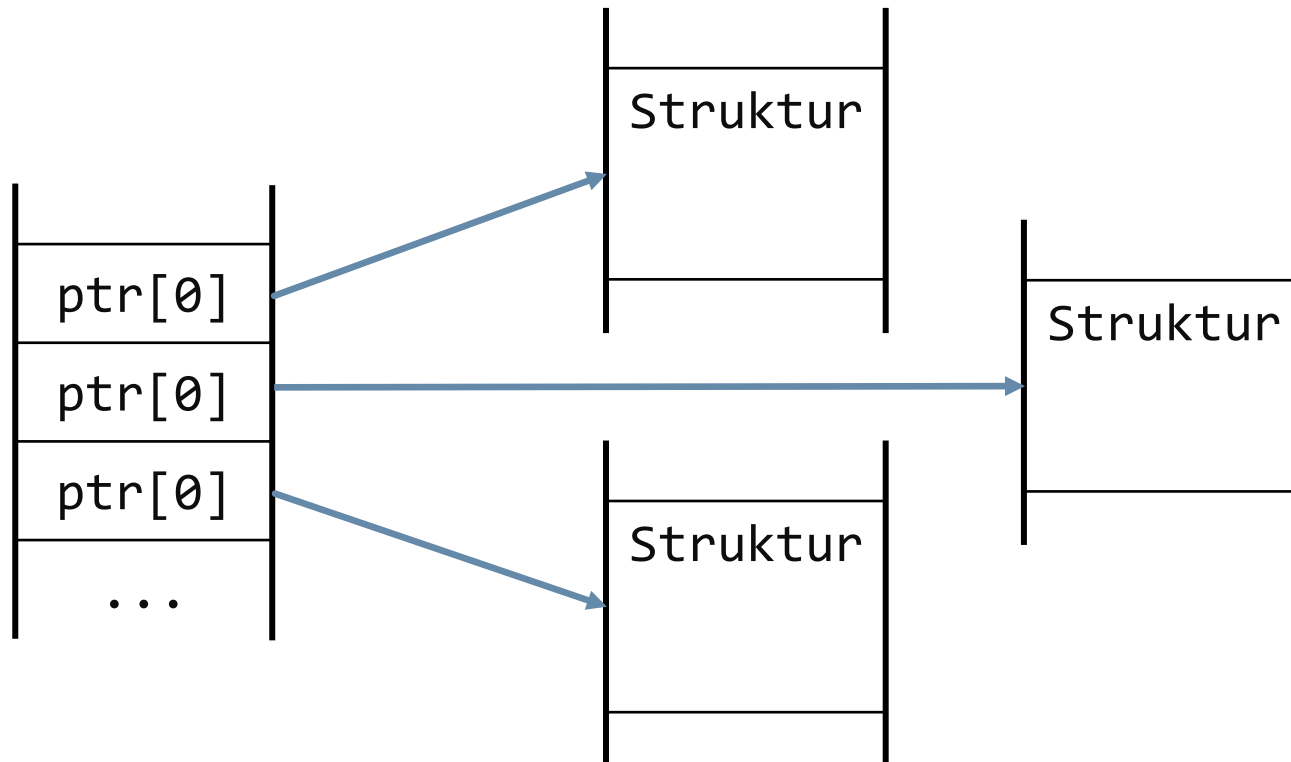
```
void func1(char** m_ptr)
```

```
{  
    *m_ptr = (char*)MEM;  
    ...  
}
```



Felder von Zeigern

Alternative zu verketteten Listen: Felder von Zeigern auf Strukturen



Felder von Zeigern

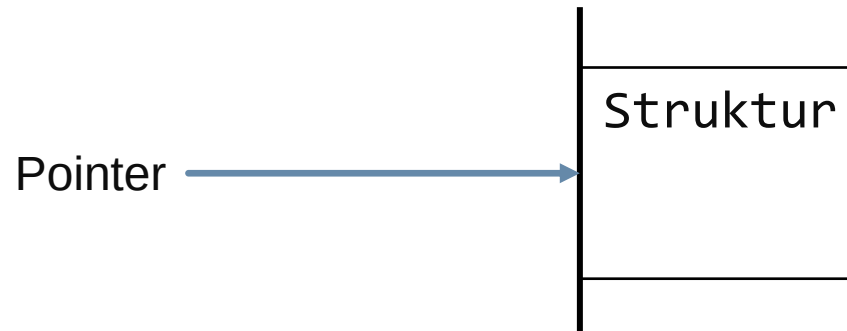
```
#define ARRAY 10
```

```
typedef struct  
{  
    long int number_der_messung;  
    float analoger_messwert;  
    ...  
} Messwert;
```

```
Messwert* mess_ptr[ARRAY];
```

```
void main(void)  
{  
    ...  
    for(loop = 0; loop < ARRAY; loop++)  
    {  
        mess_ptr[loop] = (Messwert *)malloc((size_t)(sizeof(Messwert)));  
    }  
    ...  
    mess_ptr[4]->analoger_messwert = ...;  
    ...  
}
```

Zeiger auf Funktionen



Aufruf der Funktion nicht über den Namen, sondern über den Zeiger.

Feld von Zeigern auf Funktionen:

- Auswahl der Funktion zur Laufzeit (Bestimmung des Feldindex)
- Aufruf der Funktion über ein Feldelement
- siehe auch: Exception Vector Table

Zeiger auf Funktionen – Möglichkeit 1

```
// constants
#define ...

// prototypes
void main(void);
void funk1(void);

// modul-variables
void(*fkt_ptr)();

// start of main
void main(void)
{
    ...

    // Anfangsadresse der Funktion in pointer
    fkt_ptr = funk1;

    // Funktionsaufruf
    (*fkt_ptr)();
    ...
}
// end of main

// start of function
void funk1(void)
{
    ...
}
```

Zeiger auf Funktionen – Möglichkeit 2

```
// constants
#define ...

// prototypes
void main(void);
void funk1(void);

// modul-variables
void(*fkt_ptr[10])();
```

```
// start of main
void main(void)
{
    ...

    // Anfangsadresse der Funktion in pointer
    fkt_ptr[5] = funk1;

    // Funktionsaufruf
    (*fkt_ptr[5])();
    ...
}
// end of main
```

```
// start of function
void funk1(void)
{
    ...
}
```

Zeiger auf Funktionen – Möglichkeit 3

```
// constants
#define FKT_ADR 0x20000000
```

```
// prototypes
void main(void);
void funk1(void);
```

```
// modul-variables
void(*fkt_ptr)();
```

```
// start of main
void main(void)
{
```

```
...
```

```
// Anfangsadresse der Funktion in pointer
fkt_ptr = (void(*)())FKT_ADR;
```

```
// Funktionsaufruf
(*fkt_ptr)();
```

```
...
```

```
}
// end of main
```

```
// function at address 0x20000000
// start of function
```

```
void funk1(void)
{
    ...
}
```

Zeiger auf Funktionen – Möglichkeit 4

```
// constants
#define ...

// prototypes
void main(void);
void funk1(int, float);

// modul-variables
void(*fkt_ptr)(int, float);
```

```
// start of main
void main(void)
{
    ...

    // Anfangsadresse der Funktion in pointer
    fkt_ptr = funk1;

    // Funktionsaufruf
    var1 = (*fkt_ptr)(50, 3.14);
    ...
}
// end of main
```

```
// function at address 0x20000000
// start of function
int funk1(int a, float b)
{
    ...
}
```